



SGLang Omni · 每日 PR 学习日报

报告日期：2026-06-15 | 覆盖窗口：过去 24 小时合并 (merged) 的 PR | 仓库：[sgl-project/sglang-omni](#) (SGLang Omni: High-Performance Multi-Stage Pipeline Framework for Omni Models)

窗口内合并 PR：10 个

深度讲解：5 个核心 + 1 组 CI

主线主题：Omni 多阶段调度 · TTS 服务化 · 多模态稳定性

当日概览

过去 24 小时内 main 分支共合并 10 个 PR。本仓库是一个为 **Omni**（全模态）模型服务的「多阶段流水线（**Multi-Stage Pipeline**）」推理框架——一个请求会依次/并行流经多个模型阶段（例如 **thinker** 负责理解+生成文本/语义、**talker** 负责把语义转成语音 token、**code2wav/vocoder** 负责把 token 还原成波形）。今天的改动恰好覆盖了这条流水线的几个关键环节。

Omni 调度 / mixed-chunk

TTS 服务化 / OpenAI 兼容 API

MOSS-TTS 缓存与流式

多模态 / 视频 Talker 稳定性

CI 基础设施 / 校准

本报告遵循「深度优先于广度」原则：对架构相关的核心 PR 逐行讲透，对同主题的小型 PR 做合并讲解，对纯 CI 校准类做简述。

#	标题	作者	主题	讲解深度
#789	qwen3-omni : thinker 路径默认开启 mixed-chunk	luojiaxuan	Omni 调度	深度精讲
#659	[TTS] 新增 OpenAI 兼容的 /v1/audio/speech 语音合成 API	Ratish1	TTS 服务化	深度精讲
#767	修复 Video-AMME Talker 的张量设备错配与 CI 稳定性	MelodyyyYin	多模态稳定性	深度精讲
#787	修复 MOSS-TTS Local 无 stream outbox 时的输出收集崩溃	Ratish1	TTS / 健壮性	合并精讲
#788	[Perf] 提高 MOSS-TTS Local 参考音频缓存条目上限	SandyLuXY	TTS / 缓存	合并精讲
#790	Omni 单测 CI 提前 + 修复 profiler 路由测试	Ratish1	CI / 测试设计	中等
#793-796	CI 校准 / Host Profiles / 预检清单 / 校准技能	zhaochenyang20	CI 基础设施	简述（一组）

#789 feat(qwen3-omni) : 在 **thinker** 路径上默认开启 **mixed-chunk**

merged

★ 今日最值得读

作者：luojiaxuan 合并者：zhaochenyang20 改动量：+17 / -2, 1 个文件 [查看 PR](#)

关联 #760

PR 概览

一句话：给 Qwen3-Omni 的 thinker (理解+生成阶段) 默认打开 `enable_mixed_chunk=True` 和 `chunked_prefill_size=8192`，让「正在解码的请求」不再被「正在做 prefill 的请求」饿死，并发流式翻译场景下吞吐 **+14%**、计算感知延迟 **-6%**，质量 (BLEU) 持平。

这是个「四两拨千斤」的 PR——核心改动只有 3 行，但背后是对调度器 (scheduler) 工作节律的深入观测。

它解决了什么问题

SGLang 的 thinker 调度器每一个 step (调度步) 只能二选一：要么做 **prefill**，要么做 **decode** (论文/工程里叫 "prefill XOR decode")。在并发流式场景下 (比如 32 路同声传译会话同时进行)，新请求不断到来，于是大量 step 被 "纯 prefill" 占据；而那些已经在生成 token 的请求 (decode) 只能干等这一步结束。

作者用一个 env 开关把调度器内部探针打开，量化了这个痛点 (Qwen3-Omni-30B-A3B, TP=2, 32 路流式)：

- ~98% 的纯 prefill step，其实运行队列 `running_batch` 里都有「已经可以 decode 的请求」被跳过 (平均每步约 18 个被晾着)；
- decode 只在约 58% 的调度迭代里真正推进。

结论很关键：这不是「请求不够多/到达太慢」(arrival-bound)，而是一个「占空比/节律问题 (duty-cycle problem)」——机器有活儿干，却把活儿排在了错误的时序里。

类比：把调度器想成一个只有一条收银通道的超市。每个 step 收银员要么“给一位拿着满满购物车的新顾客扫码结账” (prefill)，要么“给已经在装袋的老顾客找零” (decode)。流式高并发时，新顾客源源不断，收银员一直在扫新车，结果一堆只差找零就能走人的老顾客被堵在通道里。**mixed-chunk** 相当于让收银员“扫新车的间隙顺手把旁边老顾客的零钱找了”，通道不再空转。

具体做了什么改动

只改了一个文件：sglang_omni/models/qwen3_omni/stages.py 里的 `create_sglang_thinker_executor_from_config()`——这是构造 thinker 调度器 (OmniScheduler) 的工厂函数。改动就是把默认的 `overrides` 字典多塞两个键：

```
# 改动前：只关掉一个开关
overrides: dict[str, Any] = {"disable_cuda_graph": False}

# 改动后：额外打开 mixed-chunk，并把 chunked_prefill_size 定为 8192
overrides: dict[str, Any] = {
    "disable_cuda_graph": False,
    "enable_mixed_chunk": True,          # ← 关键：允许把 decode 折进 prefill 的
    "chunked_prefill_size": 8192,      # ← 必要条件：chunk 大小 > 0 时 mixed-ch
}

if server_args_overrides:
    overrides.update(server_args_overrides)  # ← 用户传的覆盖项仍然优先，可
overrides["tp_size"] = tp_size
```

逐点解释这段逻辑在做什么：

- **为什么两个键要一起设？** `mixed-chunk` 的机制依赖“分块 prefill”——只有当 `chunked_prefill_size > 0` 时，prefill 才会被切成固定大小的 chunk（这里是 8192 token），调度器才有“空位”把正在 decode 的请求塞进同一个前向（forward）批次。所以 `enable_mixed_chunk=True` 必须配上一个正的 chunk size 才真正起效。
- **为什么是 8192？** 作者扫了 {4096, 8192, 16384}，吞吐都在 ~9.2–9.3 seg/s 持平，8192 作为“安全默认值”。
- **会不会破坏用户配置？** 不会。`overrides.update(server_args_overrides)` 这行保证：用户在 `server_args_overrides` 或命令行 (`--enable-mixed-chunk/--chunked-prefill-size`) 里显式传的值会覆盖这里的默认值，想退出只需传 `enable_mixed_chunk=False`。同时 `cli/serve.py` 和 `qwen3-omni` 的 `factory_args` 都没设这两个键，所以默认值不会被悄悄冲掉。
- **影响范围严格收窄：**只动 Qwen3-Omni 的 thinker；ming-omni 有自己的同名工厂函数，不受影响；talker 完全没碰。

指标 (en→zh 流式同传, ACL6060)	baseline	mixed-chunk 默认	变化
吞吐 (seg/s)	~8.1	~9.3	+14%
StreamLAAL_CA (计算感知延迟)	~2280	~2143	-6%
BLEU (翻译质量)	32.1	32.1	持平

"晾着 decode"的纯 prefill 步占比	~98%	~0%	—
---------------------------	------	-----	---

涉及的知识点（重点展开）

Prefill 与 Decode（预填充 / 解码）

是什么：大模型推理分两个阶段。**Prefill** = 把整段输入 prompt 一次性喂进模型，算出每个 token 的隐状态并填充 KV cache，是一次"大批量并行"运算；**Decode** = 之后逐个 token 自回归生成，每步只算 1 个新 token，是"小而频繁"的运算。

为什么需要区分：两者算力特征完全不同——prefill 是计算密集（吃 GPU 算力），decode 是访存密集（吃显存带宽，算力利用率低）。调度器如何编排它们，直接决定吞吐和延迟。

本 PR 里：痛点正是"prefill 步会把 decode 步挤掉"。mixed-chunk 让一个 step 同时干两种活，提高了 GPU 的有效利用。

Scheduler（调度器）与 running_batch

是什么：SGLang 引擎的"大脑"。它维护一个等待队列和一个 running_batch（正在跑的请求批次），每个 step 决定"这一拍把哪些请求、做什么操作"打包成一次 GPU 前向。

为什么需要它：GPU 一次前向能处理一批 token 最划算（见下方 continuous batching）。调度器要在"延迟"和"吞吐"之间动态权衡。

本 PR 里：作者直接观测调度器：发现 running_batch 里有大量 decode-ready 请求在 prefill 步被"passed over（跳过）"，这是定位问题的核心证据。

Continuous Batching（连续批处理）

是什么：传统批处理要等一批请求全部到齐、全部结束才换下一批；连续批处理允许请求"随到随进、随完随出"，每个 step 动态重组批次。

为什么需要它：LLM 请求长度差异极大，固定批会让短请求等长请求，GPU 大量空转。连续批处理是现代推理引擎（vLLM/SGLang/TensorRT-LLM）吞吐高的根本原因。

本 PR 里：mixed-chunk 是连续批处理之上的进一步优化——不仅请求级别动态混合，连"prefill 块"和"decode 步"也混进同一个前向。

Chunked Prefill（分块预填充）

是什么：把一个超长 prompt 的 prefill 切成若干固定大小（如 8192 token）的块，分多个 step 完成，而不是一次性塞进去。

为什么需要它：(1) 超长 prompt 一次 prefill 会造成巨大的激活显存尖峰、并长时间独占 GPU 让所有 decode 停摆（延迟尖刺）；(2) 切块后留出的"空隙"正好可以塞 decode。

本 PR 里：chunked_prefill_size=8192 是 mixed-chunk 的前置条件——没有切块就没有空隙去混合 decode。对短 prefill 它是 no-op（无副作用）。

Mixed-Chunk（混合分块） **本 PR 主角**

是什么：把"正在 decode 的请求"的那 1 个 token，**折叠进**同一拍的 chunk-prefill (extend) 批次里一起算。于是这一个 GPU 前向既推进了某个长 prompt 的一块 prefill，又顺手把一堆 decode 各往前走了一步。

为什么需要它：消灭"纯 prefill 步饿死 decode"的占空比浪费。原本 decode 只在 58% 的 step 推进，开启后纯 prefill 步晾着 decode 的比例从 ~98% 降到 ~0%。

本 PR 里：就是默认把它打开。注意它**不改变数学结果**（BLEU 持平），只改变"什么时候算"，是纯粹的调度/吞吐优化。

KV Cache（键值缓存）

是什么：注意力机制里，每个已生成 token 的 Key/Value 向量会被缓存复用，这样 decode 第 N 个 token 时不必重算前 N-1 个。

为什么需要它：没有它，自回归生成是 $O(N^2)$ 重复计算；有了它每步只算新 token，是 LLM 推理可用的基础。它也是显存的主要消耗者。

本 PR 里：prefill 阶段填充 KV cache，decode 阶段读取它。mixed-chunk 把两类访问编排进同一前向，KV cache 的读写仍各自独立、结果不变。

Tensor Parallelism（张量并行，TP） & CUDA Graph

TP 是什么：把一个大模型的权重矩阵按列/行切到多张 GPU 上（这里 TP=2 = 2 卡），每卡算一部分再 all-reduce 汇总。用于单卡放不下的大模型。

CUDA Graph 是什么：把一串固定的 GPU kernel 调用"录制"成一张图，之后重放 (replay) 整张图，省去每个 kernel 逐个 launch 的 CPU 开销。decode 步形状固定，最适合用它。

本 PR 里：默认 disable_cuda_graph=False（即启用 CUDA Graph）；30B 模型用 TP=2 部署。这些是 mixed-chunk 收益得以兑现的运行环境。

我能学到什么 / 延伸思考

- "先量化、再下结论"的工程方法论：作者没有凭感觉调参，而是写了个 env-gated 探针去测"纯 prefill 步里有多少 decode 被跳过"，把问题精确归类为 duty-cycle 而非 arrival-bound。值得复用：优化前先用数据证明瓶颈在哪。
- "安全的默认值"设计：新默认值必须 (a) 可被用户覆盖、(b) 不破坏现有配置、(c) 在边界条件 ($N=1/N=8$ 低并发) 无回归、(d) 对参数不敏感 (扫了 3 个 chunk size)。这是把一个实验性优化"默认化"的标准清单。
- **A/B** 的严谨性：交换 A/B 顺序 (mixed-first) 复测，排除"先跑的占便宜"这类系统偏差。
- 前置知识建议：想吃透可以再看 → SGLang/vLLM 的 continuous batching 论文 (Orca)、chunked prefill (Sarathi-Serve 论文)、以及 prefill/decode disaggregation 的取舍。

#659

[TTS] 新增 **OpenAI** 兼容的 `/v1/audio/speech` 语音合成 API

merged

★ 服务化重头戏

作者：**Ratish1** 合并者：zhaochenyang20 改动量：+3092 / -1505，48 个文件 [查看 PR](#)

关联 #601

PR 概览

一句话：给 TTS（文本转语音）服务接上一层 **OpenAI** 风格的 `/v1/audio/speech` 接口，让任何写过 OpenAI 语音 SDK 的开发者，几乎零成本就能调用 SGLang Omni 的语音合成——支持非流式（一次返回整段音频）和 SSE 流式（边合成边吐音频块）。

它解决了什么问题

在这个 PR 之前，框架内部已经有完整的 TTS 模型流水线（能把文本变成音频），但对外没有一个标准、好用的 **HTTP** 接口。下游开发者要接入就得了解框架内部的 `GenerateRequest` 私有协议、自己处理音频编码格式、自己拼流式协议——心智负担重、迁移成本高。

OpenAI 的 `/v1/audio/speech` 已经是事实标准，生态里大量客户端/SDK 都按它来。“兼容 **OpenAI**”意味着：用户把 `base_url` 一改就能跑，几乎不用改业务代码。这就是这个 PR 的价值——把强大的内部能力包装成一个生态通用的门面。

类比：内部流水线像一台专业级咖啡机，但只有懂行的人会用一堆旋钮。这个 PR 给它装了一个“星巴克点单台”——你只要说“一杯拿铁（input + voice + 格式）”，后台自动翻译成机器的精确参数。

具体做了什么改动

整体架构（PR 自带的流程图）：

```
Client → POST /v1/audio/speech
      → CreateSpeechRequest（请求字段校验）
      → SpeechService（校验 + "降级/lowering" 转换层）
      → GenerateRequest（框架内部统一请求）
      → Pipeline client（流水线客户端）
```

→ TTS model stages (实际的多阶段 TTS 模型)
→ Audio response → Client

关键改动逐块讲：

- **CreateSpeechRequest** (请求模型)：定义对外暴露的字段——合成文本 input、音色别名 voice、响应格式 response_format (wav/pcm 等)、参考音频/文本 (用于声音克隆)、生成参数、流式控制。这是 API 的"契约"。
- **SpeechService** (校验 + **lowering** 层)：这是设计上的核心。它负责把"用户友好的 OpenAI 风格请求"翻译/降级 (**lower**) 成"框架内部的 GenerateRequest"：规范化响应格式、任务类型、语言、本地/data/HTTP 三种媒体引用方式，并产生 OpenAI 风格的校验错误。它把 **API** 表层和流水线内部解耦——内部协议怎么演进，对外契约不变。
- 服务端路由：在 OpenAI 兼容 server 里新增 /v1/audio/speech 的两条路径——非流式返回编码后的整段音频；流式用 SSE 分块返回。
- 流式约束为 **PCM**：流式只允许 PCM 格式，通过 SSE 逐块 (audio.speech.chunk 事件) 下发，这样不必改动 **pipeline client** 的契约就能暴露"增量音频"。最后还会发一个带 usage 元数据的终止块和 data: [DONE]。
- **initial_codec_chunk_frames** 打通：把这个参数从 API 一路接到 GenerateRequest.extra_params → Higgs 流式元数据 → Higgs 流式 vocoder，让"第一个音频块多大"真正可由请求控制 (以前只是文档里写写、并没生效)。
- **--allowed-local-media-path** 安全开关：之前 file:// 形式的参考音频是全局接受的 (有路径穿越/读任意文件的风险)。改为必须由部署方显式开启白名单路径才允许，默认关闭。这是一处重要的安全收敛。
- 测试 & 文档：新增针对请求校验、错误响应、流式格式强制、参考校验、请求 lowering 的单测 (test_speech_protocol.py 等共 39 passed)，并更新了用户/开发者文档。

涉及的知识点 (重点展开)

TTS / Vocoder / Codec Token (语音合成基础三件套)

是什么：**TTS**=文本转语音。现代神经 TTS 常分两步：模型先生成离散的 **codec token** (一串代表声音的整数码，像"声音的拼音")，再由 **vocoder** (声码器) 把这些 token 还原成真实波形 (wav)。

为什么需要它：直接生成原始波形 (每秒上万个采样点) 太难太慢；先生成压缩的离散 token、再解码成波形，既高效又可控。

本 PR 里：response_format 决定最终编码 (wav/pcm)；
initial_codec_chunk_frames 控制流式时第一批 codec 帧的大小；vocoder 在流水线末端把 token 变成可播放音频。

SSE (Server-Sent Events, 服务器推送事件)

是什么：一种基于 HTTP 的单向流式协议——服务器保持连接打开，持续以 `data: ... \n\n` 的文本块往客户端推数据，直到发 `data: [DONE]`。

为什么需要它：语音合成可能要几秒，用户不想等全部合成完才听到声音。SSE 让"边合成边播放"成为可能，且比 WebSocket 更轻、天然走 HTTP。OpenAI 的流式接口正是用 SSE。

本 PR 里：流式语音用 SSE 推 `audio.speech.chunk` 事件（含 `format/mime_type/sample_rate`），末尾推 `usage` 统计和 `[DONE]`。流式被限制为 PCM，因为 PCM 是裸采样、可以无损切块拼接，而 wav 有文件头不适合中途切。

"Lowering / 降级转换"（编译器借来的设计模式）

是什么："lowering"原是编译器术语——把高层、用户友好的表示，逐步翻译成低层、贴近执行的表示。这里 `SpeechService` 把高层的 `CreateSpeechRequest` 降级为低层的 `GenerateRequest`。

为什么需要它：它是一道防腐层（**Anti-Corruption Layer**）——把"对外 API 的稳定契约"和"对内引擎的易变实现"隔开。对外协议可以保持 OpenAI 兼容，对内 `GenerateRequest` 随便重构，互不污染。

本 PR 里：这正是为什么作者强调"不改动 pipeline client 契约"就能加流式——所有适配都收敛在 lowering 这一层。

OpenAI 兼容层（API 门面模式）

是什么：实现一套与 OpenAI HTTP 接口（路径、请求体、响应体、错误格式）完全一致的服务端，让客户端"无感切换"。

为什么需要它：生态护城河。OpenAI SDK 用户众多，兼容=零迁移成本接入，是开源推理框架获取用户的标准打法（vLLM、SGLang 都这么做）。

本 PR 里：新增的 `/v1/audio/speech` 字段命名（`input/voice/response_format`）和错误风格都对齐 OpenAI，curl 验证可直接 `--output speech.wav`。

本地媒体白名单 / 路径安全

是什么：对 `file://` 这类指向服务器本地文件的引用，只允许访问管理员显式列入白名单的目录。

为什么需要它：若全局放开，恶意请求可能用 `file:///etc/passwd` 之类让服务器读取/泄露任意文件（典型的 SSRF/路径穿越风险）。默认拒绝、显式开启是安全的"最小权限原则"。

本 PR 里：新增 `--allowed-local-media-path`，把"接受本地参考音频"从默认行为改成部署方主动授权的行为。

我能学到什么 / 延伸思考

- "门面 + 防腐层"是接入生态的黄金组合：对外兼容既有标准（降低用户成本），对内用 lowering 层吸收差异（保护核心演进自由）。这套模式在任何"给老引擎加新 API"的场景都适用。
- 流式协议的格式取舍：为什么流式强制 PCM 而非 wav？因为要可切块、可拼接、无文件头依赖。设计流式接口时要想清楚"哪种编码适合分块传输"。
- 把安全收敛和功能一起做：顺手把 `file://` 从默认放开改成白名单——新增能力的同时不留口子。
- 前置知识建议：OpenAI Audio API 规范、SSE vs WebSocket 的差异、神经 TTS 的 codec+vocoder 两段式架构（如 EnCodec / SoundStream / 各家 neural codec）。

#767

fix : 修复 Qwen3-Omni Video-AMME Talker 阶段的张量设备错配与 CI 稳定性

merged

★ 多模态实战排错

作者 : MelodyyyYin 合并者 : zhaochenyang20 改动量 : +124 / -110 , 13 个文件 [查看 PR](#)

关联 #765

PR 概览

一句话 : 修复 Qwen3-Omni "视频理解+说话" (Video-AMME Talker) CI 间歇性失败的一连串**真实 bug** : CPU/CUDA 张量混用导致的设备错配、TP=2 下 GPU-1 显存 OOM、跨进程传 fd 的权限错误、CI 产物 glob 不匹配、以及孤儿 GPU 进程清理。

这是一份非常典型的"多模态多阶段流水线在真实硬件上的排错日志", 每一条都对应一个值得学习的基础设施坑。

它解决了什么问题

Qwen3-Omni 的视频对话流水线在 CI 里时绿时红 (**flaky**) ——这种"间歇性失败"最折磨人, 因为它既不能说功能坏了, 又不能放心合并。作者逐个定位, 发现是 5 个独立的根因叠加, 而非单一 bug。下面挑最有代表性的讲。

具体做了什么改动

① Talker prefill 的设备错配 (最核心的代码 **bug**)

问题 : talker_prefill.py 里对一组张量做 torch.stack, 但这些张量有的在 CPU、有的在 CUDA 上, torch.stack 要求所有输入在同一设备, 于是间歇性抛错。修复就是在 stack 之前把每个张量都搬到统一设备 :

```
# 修复后 (sglang_omni/models/qwen3_omni/components/talker_prefill.py)
assistant_hidden = torch.stack(
    [
        # 关键 : 对每个 chunk 的 hidden/embed 先 .to(统一设备+dtype) 再 stack
        self.chunk_layer_hidden_or_embed(chunk).to(
            device=self._device, dtype=self._dtype,
        )
        for chunk in thinker_chunks
    ],
```

```
dim=0,  
)
```

这里 `self._device` 在初始化时被定为 `model.model.codec_embedding.weight.device`——即“模型权重在哪张卡，就以哪张卡为基准设备”。把所有参与运算的张量统一到这个设备，`torch.stack` 就不会再因设备不一致而崩。

② TP=2 下 GPU-1 的 OOM (显存超限)

在 stage 11 里，thinker 的 rank-1 和 talker_ar + code2wav 共享同一张 **GPU 1**。当 `thinker-mem-fraction-static` 设为 0.55 时，并发 16 的视频 prefill 激活尖峰把显存撑爆。修复：把它降到 0.40，并设置 `PYTORCH_ALLOC_CONF=expandable_segments:True` (让 CUDA 分配器用可扩展段、减少碎片)。细节彩蛋：PyTorch 2.9 把环境变量从 `PYTORCH_CUDA_ALLOC_CONF` 改名成 `PYTORCH_ALLOC_CONF`，所以两个名字都设以兼容新旧版本。

③ pidfd_getfd: Operation not permitted (跨进程传 fd 的权限)

请求进入“解耦式 (**disaggregated**) 中继”后，阶段进程之间需要用 Linux 的 `pidfd_getfd` 系统调用互传文件描述符 (**fd**)，这个调用需要 `CAP_SYS_PTRACE` 能力。修复：只给真正用到解耦 server 的 stage 11 容器加 `--cap-add=SYS_PTRACE` (路由/同卡共置的阶段不需要，就不加，遵循最小权限)。

④ CI 产物 glob 不匹配 & ⑤ 孤儿 GPU 进程清理

- 测试用 `mktemp("videoamme_audio")` 生成的目录实际叫 `videoamme_audio0` (带后缀)，而上传产物的 glob 没加通配符 `*`，导致从来没匹配到。修复：给 stage 4/6/8/10/11 的 glob 补 `*`。
- 重试路径会漏掉 `multiprocessing.spawn` 起的孤儿 worker——它们持有 `/dev/nvidia*` 的 fd 占着显存，却对 `nvidia-smi` 不可见。给 `delete_gpu_process.sh` 加 `--kill-orphans` (扫 `/proc/*/fd + pkill`)，并删掉已成空壳的 `ensure_gpus_idle.sh`，统一收口。

另外作者明确标注了“非本 **PR** 改动”：stage 10 的精度阈值 0.65 是预先就存在的 flaky 阈值 (模型均值 ~0.61)，不是本 PR 引入的回归，留给维护者处理。这种“划清边界”的诚实很专业。

涉及的知识点 (重点展开)

设备错配 (**device mismatch**) 与 **torch.stack**

是什么：PyTorch 张量都“住”在某个设备上 (CPU 或某张 `cuda:N`)。像 `torch.stack/cat` 这类合并操作，要求所有输入在同一设备，否则直接报 `RuntimeError`。

为什么会错配：多阶段流水线里，有的中间结果在 CPU (如刚从主机内存读出的特征)，有的在 GPU (如模型刚算出的 hidden)。混在一起就崩。

本 PR 里：统一搬到 `codec_embedding.weight.device` 这个“基准设备”再 `stack`——用“模型权重所在卡”作为唯一真相源，是个干净的约定。

Thinker / Talker / Code2wav (Omni 多阶段角色分工)

是什么：Omni 模型的典型三段式：**thinker** 理解多模态输入（文/图/视频/音）并产出语义与文本；**talker (talker_ar)** 把语义自回归地转成语音 codec token；**code2wav** 把 token 解码成波形。

为什么这样拆：每段算力/显存需求不同，拆开可以分别放在不同卡、分别优化、分别扩缩容——这正是"Multi-Stage Pipeline"框架的核心理念。

本 PR 里：OOM 正是因为 thinker rank-1、talker_ar、code2wav 被**共置 (colocate)** 在同一张 GPU 1 上抢显存——拆分带来灵活，也带来资源争用，需要精细的显存预算 (mem-fraction)。

显存预算：mem-fraction-static 与 expandable_segments

是什么：mem-fraction-static 指引引擎静态预留多少比例显存给 KV cache / 权重；expandable_segments:True 让 CUDA caching allocator 用可按需扩展的内存段，减少因碎片导致的"明明总量够却分不出大块"的 OOM。

为什么需要它：共置多个阶段时，必须给每个阶段留够、又不能各自贪多。激活尖峰 (prefill 时最大) 是最容易爆的时刻。

本 PR 里：0.55→0.40 给共置的 talker/code2wav 让出空间，配合可扩展段吸收 prefill 尖峰，OOM 消失。

Disaggregated Serving (解耦式服务) 与 fd 传递

是什么：把推理的不同部分（如 prefill 与 decode、或不同阶段）放到不同进程/节点上，用中继 (relay) 转发请求和数据。进程间有时需要直接共享打开的文件/socket，靠 Linux pidfd_getfd 把一个进程的 fd "搬"到另一个进程。

为什么需要 CAP_SYS_PTRACE：从别的进程偷取 fd 是高权限操作（等于能窥探别的进程），内核要求调用方具备 ptrace 能力，容器默认不给。

本 PR 里：只给用到解耦 server 的 stage 11 容器加这个 capability，其余阶段不加——精确授权而非全局放开。

孤儿进程 / multiprocessing.spawn 与 GPU fd 泄漏

是什么：Python 用 spawn 起的子进程若父进程异常退出，可能变成"孤儿"——还活着、还持有 /dev/nvidia* 的 fd 占着显存，但 nvidia-smi 的进程列表里可能看不到它们。

为什么棘手：下一个测试想用显存却发现被占，且找不到凶手，于是清理等待超时、把本来能过的测试判红。

本 PR 里：通过扫描 `/proc/*/fd` 找出持有 `nvidia fd` 的进程再 `pkill (--kill-orphans)`，从根上回收，让 `flaky` 变稳定。

我能学到什么 / 延伸思考

- "**flaky** 往往不是一个 **bug** 而是一堆"：本 PR 一口气修了 5 个独立根因。面对间歇性失败，要有耐心逐个隔离，而不是寄望于单点修复。
- 共置 (**colocation**) 是把双刃剑：省卡，但带来显存争用、设备错配、`fd/capability` 等一连串系统级问题。多阶段框架的难点很多在"系统/OS 层"而非"模型层"。
- 最小权限：`capability`、本地路径白名单都只在真正需要的地方开——和 #659 的安全思路一脉相承。
- 诚实划清边界：把"不是我引入的 `flaky` 阈值"明确标注、留给维护者，避免在一个 PR 里夹带不相关改动。
- 前置知识建议：`CUDA caching allocator` 与显存碎片、`Linux capabilities` 与 `pidfd` 系列系统调用、`Python multiprocessing` 的 `spawn/fork` 差异。

#787 + #788

MOSS-TTS Local : 输出收集健壮性修复 + 参考音频 缓存容量调优

merged x2

作者 : Ratish1 (#787) / SandyLuXY (#788) #787: +6/-1 | #788: +5/-5 [#787 ↗](#) · [#788 ↗](#)

概览 (两个小而精的 TTS 基础设施 PR)

#787 : 修复单测里用 `__new__` 构造 runner 时, 因缺少 `_outbox` 属性导致 `post_process_outputs()` 崩溃——让"流式发送"成为可选副作用, 不再阻塞核心的"输出行收集"。

#788 : 把 MOSS-TTS Local 的参考音频缓存条目上限从 1024 提到 8192 (字节上限 64 MiB 不变), 让"字节上限"而非"条目数"成为更大声音克隆负载下的实际约束。性能对当前负载持平 (纯容量余量改动)。

它们各自解决了什么问题

#787 的痛点 : PR #753 给 MOSS-TTS Local 加流式时, 在 `__init__()` 里初始化 `_outbox`, 并在 `post_process_outputs()` 里直接读 `self._outbox`。但很多单测故意用 `__new__`/`object.__new__` 绕过完整构造来单独测 `model-runner` 钩子。结果"可选的流式状态"变成了"输出收集的硬依赖", 一测就 `AttributeError`。

#788 的痛点 : MOSS-TTS Local 把编码后的参考音频缓存成离散 **codec token** 张量 (不是波形也不是连续特征)。SeedTTS EN 负载里 666 个唯一参考音频总共才占 ~1.85 MiB, 远低于 64 MiB 的字节上限——但若条目数上限太小, 会在条目数维度提前淘汰。

具体改动

#787 :

- 给 `MossTTSLocalModelRunner._outbox` 和 `_vocoder_target` 加类级默认值, 这样即使构造被绕过, 实例也有和正常对象一致的"无流式"默认状态。
- 把 `post_process_outputs()` 里的可选流式检查拆开: 先把非 EOS 的 journal 行 append 进 `output_rows` (这是核心契约), 再在没有 `stream_metadata` 或没配 `outbox` 时跳过流式发布。生产环境的流式路径 (`set_stream_outbox` + 发 `OutgoingMessage(type="stream", target="vocoder")`) 保持不变。

#788 : 在 `config.py` 与 `stages.py` 把默认 `ref_audio_cache_max_items` 1024→8192 (默认双卡配置和共置配置都走 `_stages(...)` 同时生效), `ref_audio_cache_max_bytes`=64 MiB 的 LRU 上限不变; 并更新单测断言三条构造路径都用 8192。

缓存行为 (**SeedTTS EN**)

cache=1024

cache=8192

最终缓存条目数	666	666
占用字节	1,850,592	1,850,592

两者命中行为完全一致，故吞吐/延迟在当前负载持平（±1% 噪声内）——这是有意为之的"容量余量"改动，为更大负载预留。

涉及的知识点

outbox / 流式路由 (scheduler outbox routing)

是什么：outbox 是调度器的"出件箱"——model runner 把要发给下游阶段（如 vocoder）的消息放进去，由调度器路由转发。流式 TTS 就是不断把 codec 块通过 outbox 推给 vocoder 实时出声。

为什么是"可选副作用"：核心契约是"runner 产出 output journal 行"；要不要把这些行同时流式推给 vocoder，取决于请求是否流式（是否配了 stream_to=["vocoder"]）。#787 的修复正是把这层"可选"在代码结构上落实。

用 __new__ 绕过构造的单测手法

是什么：object.__new__(Cls) 创建一个实例但不调用 __init__，于是不会拉起完整依赖（这里是整个 SGLang worker 栈）。

为什么用它：单测想孤立验证某个方法（如 post_process_outputs）的逻辑，不想付出启动 GPU/进程的代价。

启示：这种测试要求方法不能假设"所有实例属性都已在 __init__ 设好"。#787 用类级默认值解决——这是兼顾"可测试性"与"运行时正确性"的经典手法。

LRU 缓存的双维度上限 (条目数 vs 字节)

是什么：LRU (Least Recently Used, 最近最少使用) 缓存淘汰最久未用的项。这里同时有两个上限：条目数 max_items 和总字节 max_bytes，任一触顶就淘汰。

为什么要两个维度：条目数防止"很多小项"撑爆数量，字节数防止"少数大项"撑爆内存。设计缓存时让"更合理的那个维度"成为实际约束。

本 PR 里：参考音频是小 token 张量，字节上限远难触及，所以把条目上限抬高，让 64 MiB 字节上限成为真正的"硬预算"，避免在大声音库下因条目数过早淘汰、反复 miss。

我能学到什么

- 可选功能不要变成核心路径的硬依赖：#787 的本质教训——新增的流式状态把基础的输出收集"绑架"了。区分"核心契约"与"可选副作用"，并在代码结构上显式分离。
- 类级默认值是构造健壮性的廉价保险：给可能被绕过初始化的属性设类级默认，几行就能消除一类 `AttributeError`。
- "性能中性的容量改动"也要用数据证明：#788 明确给出"两种配置命中完全一致、字节占用相同"，证明这是纯余量、无回归——改配置也讲证据。

作者：Ratish1 改动量：+51 / -21，6 个文件 [查看 PR](#)

概览

一句话：修复一个因 FastAPI/Starlette 升级而变脆的 profiler 路由单测（改成真发请求而非检查路由对象内部结构），并把便宜的 PR 单测挪到昂贵的 TTS/Qwen3-Omni 基准测试之前先跑，同时恢复复用 venv 的依赖刷新。

它解决了什么问题

老测试 `test_launcher_uses_runner_and_mounts_profiler_routes` 直接去遍历 `app.routes` 检查路由的对象形状。但 launcher 是用 `app.include_router(router)` 挂载 profiler 端点的——新版 FastAPI/Starlette 可能把端点藏在 "included-router 条目" 后面，顶层 `app.routes` 不再逐个暴露具体路由。于是测试断言的是 "恰好成立的内部布局"，框架一升级就误报失败。

具体改动

- 把测试改成用 `TestClient` 真正请求 `/start_profile` 和 `/stop_profile`，验证 "端点能被正确分发" 这一真实契约，而不是路由对象长什么样。
- 用一个最小 **fake** 替换真实的 `ProfilerControlClient`，这样测端点分发时不必真去开 IPC socket。
- CI 顺序从 `setup` → `TTS` → `Qwen3-Omni` → `PR Test` 改为 `setup` → `PR Test` → `TTS` → `Qwen3-Omni`——便宜快的先跑，能更早 fail-fast。
- `prepare_omni_venv.sh` 复用 `venv` 的路径改用 `uv pip install --upgrade -e .`，避免缓存环境长期留着过时但 "兼容" 的依赖版本。

涉及的知识点

测 "行为契约" 而非 "实现细节"

是什么：好的测试应验证 "对外可观察的行为"（发请求能不能命中端点），而不是 "内部怎么实现的"（路由对象在列表里排第几）。

为什么：断言实现细节的测试是 "脆性测试 (brittle test)"——依赖库一升级、内部结构一变就误报，维护成本高、还会掩盖真实回归。

本 PR 里：用 `TestClient` 发真实 HTTP 请求，正是从 "检查 `app.routes` 形状" 转向 "验证端点真的能响应"。

Fail-fast 的 CI 编排 & profiler / IPC

Fail-fast : 把又快又便宜的检查放最前面，一旦挂掉立刻反馈，省下后面昂贵 GPU 基准的时间和成本。

Profiler/IPC : `/start_profile //stop_profile` 是控制性能剖析的端点；
ProfilerControlClient 通过 IPC（进程间通信）socket 给各阶段进程发指令。用 fake 替换它，测试就不必真起 socket。

我能学到什么

- 面向契约写测试能极大降低"框架升级即测试爆炸"的维护负担。
- **CI** 编排顺序本身就是一种优化：便宜的先跑、贵的后跑，是省钱省时的免费午餐。
- 复用环境要主动刷新依赖，否则缓存会悄悄留住过时版本，埋下"本地能过 CI 挂"的隐患。

#793-796

CI 基础设施：校准 / Host Profiles / 预检清单 / 校准技能（一组简述）

merged x4

这一组由 zhaochenyang20 提交的 PR 都属于 **CI** 基础设施与基准校准，与 Omni 模型运行时架构关系较弱，故合并简述：

- **#796 [CI] Calibration 2026/06/15**：当日基准阈值校准——把各 stage 的延迟/吞吐/精度门槛按最新硬件实测重新标定，减少 flaky 误报。
- **#795 [CI] Add Pre Check List**：新增提交前预检清单，规范 PR 在跑昂贵 GPU CI 前应自查的项目。
- **#794 [CI] CI Host Profiles**：引入"主机画像"概念，按不同 GPU 主机（H20/H100/H200...）记录其硬件特征，让基准阈值能按硬件分别匹配。
- **#793 [CI] Update Calibration Skills**：更新校准用的自动化技能/脚本。

为什么基准要"按硬件校准"

背景：同一段推理代码在 H20 和 H100 上的延迟/吞吐天差地别。若用一套固定阈值，换硬件就会大面积误报。Host Profiles + 周期性 Calibration 让"门槛随硬件走"，是把 GPU CI 稳定下来的工程基石——和 #767、#790 治理 flaky 是同一条主线。

今日主线串讲 (把 10 个 PR 连成一条故事线)

把今天的改动按"一个 **Omni** 请求的生命周期"串起来，会看得更清楚：

- 入口（服务化） → #659 给 TTS 接上 OpenAI 兼容门面，请求从这里进来。
- 理解 + 调度 (**thinker**) → #789 用 mixed-chunk 让 thinker 调度器别再饿死 decode，吞吐 +14%。
- 语音生成 (**talker/code2wav**) → #767 修好视频 Talker 的设备错配/OOM/fd 权限；#787 让输出收集不被流式状态绑架；#788 给参考音频缓存留足余量。
- 质量护栏 (**CI**) → #790 修脆性测试 + fail-fast 编排；#793-796 按硬件校准基准、稳住 flaky。

贯穿全天的三个工程母题：(1) 先量化再优化（#789 的调度探针、#788 的缓存证据）；(2) 最小权限 / 安全收敛（#659 本地媒体白名单、#767 仅给需要的容器加 capability）；(3) 面向契约而非实现（#659 的 lowering 防腐层、#790 的行为型测试、#787 的核心/可选分离）。

SGLang Omni 每日 PR 学习日报 · 自动生成于 2026-06-15

数据来源：GitHub [sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni) · 窗口：过去 24 小时合并的 PR

本报告侧重讲透架构相关 PR，纯 CI 校准类做简述。所有性能数据引自各 PR 描述中的作者实测。